

Neyron mashina tarjimasi: "Seq2Seq" va "Attention" mexanizmi

Tabiiy tilni qayta ishlash — 11-ma'ruza

A.A. Abdulali

11:30–12:50 · mavzu №11

2-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: bir tildan boshqasiga tarjima qilamiz
- 2 Enkoder-Dekoder va bottleneck
- 3 Attention: dekodek ortga qaraydi
- 4 Attention hisoblash: query, key, value
- 5 Tarjimani BLEU bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darslardan ikki savol

1-savol

Oldingi darslarda LSTM gapni ketma-ket o'qib, oxirgi yashirin holat h_T orqali umumiy ma'noni saqlaydi. Tarjimada manba gapni qanday eslab qolamiz?

2-savol

Butun manba gapni **bitta** vektorga siqsak, ayniqsa uzun gaplarda qanday muammo yuzaga keladi?

Takrorlash: o'tgan darslardan ikki savol

1-savol

Oldingi darslarda LSTM gapni ketma-ket o'qib, oxirgi yashirin holat h_T orqali umumiy ma'noni saqlaydi. Tarjimada manba gapni qanday eslab qolamiz?

2-savol

Butun manba gapni **bitta** vektorga siqsak, ayniqsa uzun gaplarda qanday muammo yuzaga keladi?

1-javob

Enkoder manba gapni h_T kontekstiga kodlaydi, dekoder esa shu kontekstdan tarjimani ketma-ket yaratadi. Bu yondashuv **enkoder-dekoder** arxitekturasini deyiladi.

2-javob

Boshidagi yoki uzoqdagi so'zlar haqidagi axborot susayishi mumkin. Bu **information bottleneck** (axborot torligi) muammosi. Yechim: **attention** (e'tibor mexanizmi).

Dars yakunida nimalarni bajara olasiz?

- ❶ **Tushuntira olasiz: enkoder–dekoder** (*encoder–decoder*) arxitekturasini manba gapni qanday kodlab, tarjimini ketma-ket yaratishini.
- ❷ **Izohlay olasiz: information bottleneck** (**axborot torligi**) muammosi nima uchun uzun gaplarda tarjima sifatini pasaytirishi mumkinligini.
- ❸ **Hisoblay olasiz: attention** (**e'tibor mexanizmi**) vaznlarini va kontekst vektorini:
$$\alpha = \text{softmax}([2, 1, 3]) \approx [0.245, 0.090, 0.665].$$
- ❹ **Bog'lay olasiz: query–key–value** tushunchalarini so'rov, kalit va qiymat sifatida attention hisoblash jarayoni bilan.
- ❺ **Baholay olasiz:** tarjima sifatini **BLEU** (*Bilingual Evaluation Understudy*) metrikasi bilan.

LSTM va Bi-LSTM; ketma-ketlik bilan ishlash; softmax; skalyar ko'paytma; vaznlangan yig'indi; exp funksiyasi.

Bugungi dars rejası

Bosqich	Mavzu	Natija
1	Enkoder–dekoder va <i>bottleneck</i> (axborot torligi)	O'zgarmas kontekst muammosini tushunish
2	<i>Attention</i> (e'tibor mexanizmi)	Har dekoder qadamida yangi kontekst olish
3	<i>Attention</i> hisoblash: Q/K/V (<i>query–key–value</i>)	$\alpha = \text{softmax}([2, 1, 3]) \approx [0.245, 0.090, 0.665]$
4	BLEU bilan baholash	Sodda misolda BLEU-1 ni hisoblash
–	Xulosa va 11-amaliyotga ko'rsatmalar	Attentionli Seq2Seq modeliga tayyorgarlik

Darsning asosiy zanjiri

Avval enkoder–dekoderning o'zgarmas kontekst muammosini ko'ramiz; keyin attention yordamida har qadam uchun mos kontekst quramiz. Bugungi α qiymatlari 11-amaliyotdagi hisoblashlar bilan bog'lanadi.

Bugungi yangi vazifa: ketma-ketlikdan ketma-ketlikka

Oldingi tur: teglash, 1:1

Kirishdagi **har token** uchun bitta yorliq olinadi. Kirish va chiqish uzunligi odatda **bir xil**.

Malika Toshkentda ishlaydi

→ B-PER B-LOC O

L11: tarjima, Seq2Seq, $n \rightarrow m$

Butun manba gapdan boshqa tildagi butun gap yaratiladi. Kirish va chiqish uzunligi **farq qilishi mumkin**.

“Men kitob o'qidim” (3 so'z)

→ “I read a book” (4 so'z)

Yangi savol

Endi model bitta token uchun yorliq emas, **butun gapdan butun gap** yaratishi kerak. Buning uchun **enkoder-dekoder** (*encoder-decoder*) arxitekturasini ishlatiladi. Ammo manba gapni bitta vektorga siqish **bottleneck** (**axborot torligi**) muammosini keltirib chiqaradi.

Bugungi muammo: bitta kontekst uzun gapda yetmaydi

Sodda enkoder–dekoder qayerda qiynaladi?

O'zgarmas kontekst

Enkoder butun manba gapni bitta kontekstga, masalan h_n^{enc} ga, siqadi. Dekoder har qadamda asosan shu kontekstga tayanadi.

Uzun gapda axborot torligi

Gap uzunlashganda boshidagi yoki uzoqdagi so'zlar haqidagi axborot susayishi mumkin. Natijada tarjima sifati pasayadi.

Bugungi yechim: attention

- 1 Enkoder har manba tokeni uchun alohida holat saqlaydi: $h_1, \dots, h_n$.
- 2 Dekoder har qadamda kerakli manba tokenlariga ko'proq e'tibor beradi.
- 3 Attention vaznlari $\alpha_{t,i}$ topiladi va shu vaznlar orqali yangi kontekst c_t hosil qilinadi.

Asosiy g'oya

Attention dekoderga har qadamda manbaning tegishli qismiga qayta qarash imkonini beradi. Mashina tarjimasida bu g'oya Bahdanau attention nomi bilan mashhur.

Prezentatsiya rejasi

- 1 Kirish: bir tildan boshqasiga tarjima qilamiz
- 2 **Enkoder-Dekoder va bottleneck**
- 3 Attention: dekodeer ortga qaraydi
- 4 Attention hisoblash: query, key, value
- 5 Tarjimani BLEU bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: o'qi, kodla, keyin tarjima qil

Tarjima ikki bosqichda bajariladi:

- **Enkoder** (LSTM): manba gapni ketma-ket o'qiydi va uni kontekstga kodlaydi.
- **Dekoder** (LSTM): shu kontekstdan boshlab, maqsad tilda so'zlarni ketma-ket yaratadi.

[men kitob o'qidim] $\rightarrow c \rightarrow$ [I read a book]

O'qitishda dekoderda oldingi etalon so'z beriladi (*teacher forcing*);
qo'llash bosqichida esa model o'zining oldingi chiqishiga tayanadi.

Cheklov

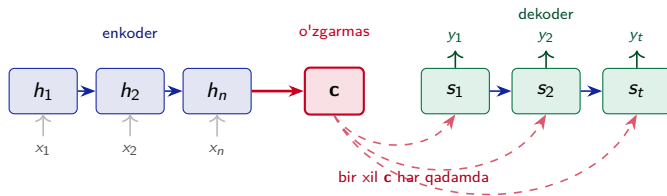
Sodda enkoder-dekoderda kontekst c bitta o'zgarmas vektor sifatida ishlatiladi. Uzun gaplarda butun manba axborotini bitta vektorda saqlash qiyinlashadi. Bu **bottleneck** (**axborot torligi**) muammosiga olib keladi.

Enkoder–dekoder: rasmiy ta'rif

$$\mathbf{c} = \mathbf{h}_n^{enc}, \quad \mathbf{s}_t = \text{LSTM}_{dec}(\mathbf{s}_{t-1}, [\mathbf{e}(y_{t-1}); \mathbf{c}]), \quad \mathbf{p}_t = \text{softmax}(\mathbf{W}_o \mathbf{s}_t + \mathbf{b}_o)$$

bunda: $\mathbf{h}_n^{enc}$ — enkoderning oxirgi holati; $\mathbf{c}$ — o'zgarmas kontekst (*fixed context*); $\mathbf{e}(y_{t-1})$ — oldingi maqsad so'z embeddingi; $\mathbf{s}_t$ — dekoder holati; $\mathbf{p}_t$ — lug'at bo'yicha ehtimollar taqsimoti.

Diqqat: **bir xil c** barcha dekoder qadamlari uchun ishlatiladi. Uzun gaplarda aynan shu holat **bottleneck** (**axborot torligi**) muammosini keltirib chiqaradi.



Nega o'zgarimas kontekst yetmaydi?

Attention qo'llanmaganda butun manba gap bitta kontekstga siqiladi:

$$\mathbf{c} = \mathbf{h}_n^{enc}, \quad \dim(\mathbf{c}) = d \text{ o'zgarimas.}$$

- ❶ **1-qadam.** Gap uzunligi oshsa ham kontekst vektori o'lchami o'zgarmaydi.
- ❷ **2-qadam.** Ko'proq axborot bir xil o'lchamga siqiladi; uzoqdagi yoki boshidagi so'zlar haqidagi axborot susayishi mumkin.
- ❸ **3-qadam.** Dekoder har qadamda bir xil $\mathbf{c}$ ni ko'radi; shu sababli ayni vaqtda qaysi manba tokeni muhimligini bevosita tanlay olmaydi.

Xulosa

Har dekodeer qadami uchun **mos kontekst** kerak. Attention (**e'tibor mexanizmi**) shu kontekstni manba holatlaridan vaznlab hosil qiladi.

Qo'lda misol: o'zgarmas kontekstning cheklovi

$h_1 = [1, 0]$ (men), $h_2 = [0, 1]$ (kitob), $h_3 = [1, 1]$ (o'qidim).

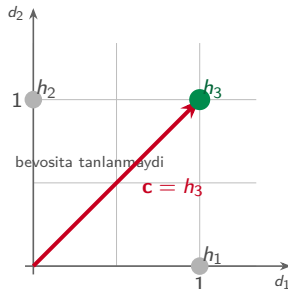
Attention bo'lmaganda:

$$\mathbf{c} = h_3 = [1, 1]$$

Dekoder barcha qadamlarda bir xil $\mathbf{c}$ ga tayanadi.
Demak, h_1 va h_2 dagi axborot **alohida tanlanmaydi**.

Nega muammo?

Agar “men” so'ziga mos axborot kerak bo'lsa, h_1 muhim. Ammo o'zgarmas kontekst $\mathbf{c} = h_3$ bu ehtiyojni bevosita ajratib bermaydi.



Kontekst h_3 ga teng bo'lib qoladi; shu sababli kerakli manba holatini alohida ajratish qiyinlashadi.

Sizning vazifangiz: o'rtacha kontekst

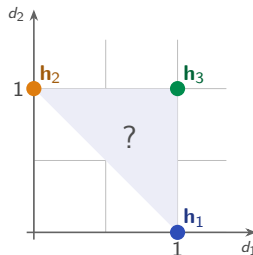
Savol

Ba'zan kontekst barcha holatlarning **o'rtachasi** sifatida olinadi: $\mathbf{h}_1 = [1, 0]$, $\mathbf{h}_2 = [0, 1]$, $\mathbf{h}_3 = [1, 1]$.

$$\mathbf{c}_{\text{avg}} = \frac{1}{3}(\mathbf{h}_1 + \mathbf{h}_2 + \mathbf{h}_3) = ?$$

Uch holatni grafikda belgilaymiz.

Ularning **markazi** (o'rtachasi) qayerda bo'ladi?



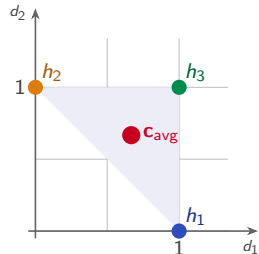
Markaz qayerda?

Sizning vazifangiz: o'rtacha kontekst

Javob

$$\mathbf{c}_{\text{avg}} = \frac{1}{3} ([1, 0] + [0, 1] + [1, 1]) = \frac{1}{3} [2, 2] \approx [0.667, 0.667].$$

O'rtacha kontekst **o'zgarmas**: har bir holatga teng vazn $\frac{1}{3}$ beriladi, shu bois qaysi so'z muhimligini ajratmaydi. Attention esa vaznlarni **har xil** tanlay oladi.



Teng vazn $\Rightarrow$ markaz (centroid).

Kod ↔ formula: enkoder-dekoder skeleti

```
import torch.nn as nn
class Seq2Seq(nn.Module):
    def __init__(self, src_v, tgt_v, dim, hid):
        super().__init__()
        self.emb_s = nn.Embedding(src_v, dim)
        self.emb_t = nn.Embedding(tgt_v, dim)
        self.enc = nn.LSTM(dim, hid, batch_first=True)
        self.dec = nn.LSTM(dim, hid, batch_first=True)
        self.out = nn.Linear(hid, tgt_v)
    def encode(self, x):
        # x: manba token indekslari
        _, (h_n, c_n) = self.enc(self.emb_s(x))
        return h_n, c_n  # dekoder boshlang'ich holati
```

Kod → formula:

Kod	Mazmuni
emb_s	manba embeddingi $e(x_i)$
enc	enkoder LSTM
h_n	oxirgi yashirin holat
c_n	LSTM cell state
dec	dekoder LSTM
out	lug'atga proyeksiya

Eslatma

PyTorchdagi `c_n` — LSTM cell state. Oldingi slayddagi `c` esa umumiy o'zgarmas kontekst ma'nosida edi.

11-amaliyotda shu skelet ustiga attention qo'shiladi.

Bottleneck tuzog'i: uzun gapda sifat pasayishi mumkin

Uzunlik bilan qiyinchilik ortadi

Attention qo'llanmagan Seq2Seq qisqa gaplarda ishlashi mumkin. Ammo uzun gaplarda manba axborotini **bitta o'zgarmas vektorga** siqish qiyinlashadi.

Natijada model ayrim manba so'zlarini yetarli saqlay olmaydi va tarjima sifati, masalan BLEU ko'rsatkichi, pasayishi mumkin.

Yechimga o'tish

Attention dekoderga har qadamda manbaning tegishli qismiga qayta murojaat qilish imkonini beradi.

Bu g'oya mashina tarjimasida Bahdanau va hammualliflar ishlari orqali mashhur bo'lgan.

Dekoder qaysi manba holatiga qarashni qanday hisoblaydi? Keyingi bo'limda attention vaznlari $\alpha_{t,i}$ ni ko'ramiz.

Prezentatsiya rejasi

- 1 Kirish: bir tildan boshqasiga tarjima qilamiz
- 2 Enkoder-Dekoder va bottleneck
- 3 Attention: dekodeer ortga qaraydi**
- 4 Attention hisoblash: query, key, value
- 5 Tarjimani BLEU bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: har so'zni yozayotganda kerakli joyga qaraymiz

Oddiy intuitsiya:

Inson tarjimon maqsad tilda har bir so'zni yozayotganda manba gapdagi **tegishli** so'z yoki iboraga qayta qaraydi.

Attention ham shu g'oyani modellaydi: dekoder t -qadamda enkoder holatlariga **e'tibor vazni** beradi.

$$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i$$

Bu yerda $\alpha_{t,i}$ — t -dekoder qadamida i -manba holatiga berilgan vazn.

Asosiy farq

O'zgarmas kontekst o'rniga har qadamda **yangi kontekst** $\mathbf{c}_t$ hosil qilinadi.

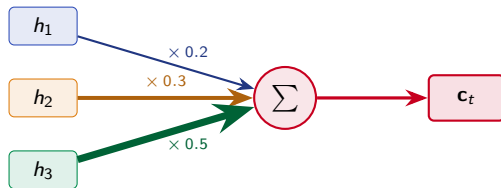
Vaznlar $\alpha_{t,i}$ qaysi manba tokeni muhimligini ko'rsatadi. Bu manba va tarjima orasidagi **moslik** (*alignment*) sifatida talqin qilinadi.

Attention konteksti: rasmiy ta'rif

$$\mathbf{c}_t = \sum_{i=1}^n \alpha_{t,i} \mathbf{h}_i, \quad \sum_{i=1}^n \alpha_{t,i} = 1, \quad \alpha_{t,i} \geq 0$$

bunda: $\mathbf{h}_i$ — i -enkoder holati; $\alpha_{t,i}$ — t -dekoder qadamida i -manba tokeniga berilgan e'tibor vazni; $\mathbf{c}_t$ — shu qadam uchun vaznlangan kontekst.

Dekoder endi har qadamda yangi $\mathbf{c}_t$ bilan ishlaydi. Vaznlarni qanday topishni keyingi bo'limda ko'ramiz.



Har $\mathbf{h}_i$ o'z vazni $\alpha_{t,i}$ ga ko'paytiriladi ($\uparrow$ vazn $= \uparrow$ qalin strelka = ko'proq hissa), so'ng yig'iladi.

Keltirib chiqarish: o'zgarmas kontekstdan vaznlanganga

- ① **Attention qo'llanmaganda:** butun manba gap bitta o'zgarmas kontekstga siqiladi:

$$\mathbf{c} = \mathbf{h}_n^{enc}.$$

- ② **Umumlashtiramiz:** kontekst faqat oxirgi holat emas, balki barcha enkoder holatlarining vaznlangan yig'indisi bo'lishi mumkin:

$$\mathbf{c}_t = \sum_{i=1}^n \alpha_{t,i} \mathbf{h}_i.$$

- ③ **Vaznlar sharti:** $\alpha_{t,i} \geq 0$ va $\sum_{i=1}^n \alpha_{t,i} = 1$. Shuning uchun ular odatda **softmax** orqali olinadi.

O'zgarmas kontekst attentionning maxsus holati sifatida qaralishi mumkin: agar har qadamda $\alpha_{t,n} = 1$ va qolgan vaznlar 0 bo'lsa, unda $\mathbf{c}_t = \mathbf{h}_n^{enc}$ bo'ladi.

Xulosa

Attention kontekstni **moslashuvchan** qiladi: dekoder har qadamda manbaning kerakli qismiga ko'proq e'tibor bera oladi.

Qo'lda misol: vaznlangan kontekst

Soddalik uchun bitta qadamdagi attention vaznlarini olaylik:

$$h_1 = [1, 0], \quad h_2 = [0, 1], \quad h_3 = [1, 1], \quad \alpha_1 = 0.2, \alpha_2 = 0.3, \alpha_3 = 0.5.$$

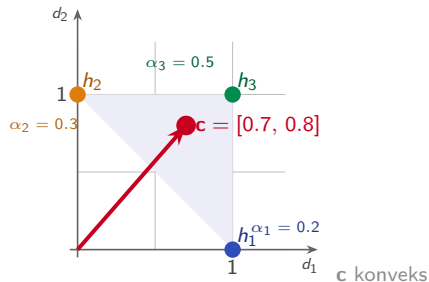
Hisoblaymiz:

$$\mathbf{c} = 0.2[1, 0] + 0.3[0, 1] + 0.5[1, 1]$$

$$\mathbf{c} = [0.2 + 0.5, 0.3 + 0.5] = [0.7, 0.8].$$

Xulosa

Eng katta vazn 0.5 bo'lgani uchun $\mathbf{c}$ nuqta h_3 tomonga ko'proq siljiydi. Demak, attention vazn-lari kontekst qaysi manba holatiga ko'proq tayan-ishini boshqaradi.



kombinatsiya bo'lgani uchun uchburchak ichida joylashadi va h_3 tomon ko'proq tortiladi.

Sizning vazifangiz: boshqa vaznlar bilan kontekst

Savol

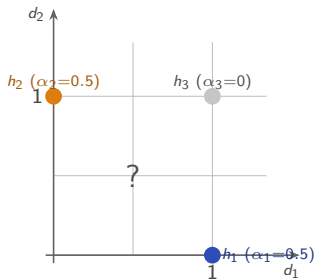
$h_1 = [1, 0]$, $h_2 = [0, 1]$, $h_3 = [1, 1]$; vaznlar:

$$\alpha_1 = 0.5, \quad \alpha_2 = 0.5, \quad \alpha_3 = 0.$$

$\mathbf{c} = ?$

Uchta holat va ularning vaznlari berilgan.

$\mathbf{c}$ vektorini hisoblang va grafikda **qayerga** tushishini aniqlang. E'tibor bering: $\alpha_3 = 0$.



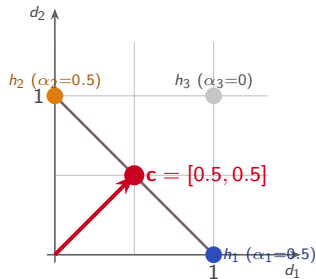
$\mathbf{c}$ qayerga tushadi?

Sizning vazifangiz: boshqa vaznlar bilan kontekst

Javob

$$\mathbf{c} = 0.5[1, 0] + 0.5[0, 1] + 0[1, 1] = [0.5, 0.5].$$

E'tibor h_1 va h_2 ga teng taqsimlanadi, h_3 esa ($\alpha_3 = 0$) kontekstga hissa qo'shmaydi. Shuning uchun $\mathbf{c}$ nuqta h_1 – h_2 kesmasining o'rtasida yotadi.



$\mathbf{c}$ — h_1 – h_2 kesmasi o'rtasida.

Kod $\leftrightarrow$ formula: vaznlangan kontekst

```
import numpy as np

H = np.array([[1, 0], # h_1
              [0, 1], # h_2
              [1, 1]]) # h_3

alpha = np.array([0.2, 0.3, 0.5])

# c_t = sum_i alpha_i h_i
context = alpha @ H

print(context) # [0.7 0.8]
```

Kod $\rightarrow$ formula:

Kod	Formula / ma'no
H	$\mathbf{H} = [\mathbf{h}_1; \mathbf{h}_2; \mathbf{h}_3]$
alpha	$\alpha_t = [\alpha_{t,1}, \alpha_{t,2}, \alpha_{t,3}]$
alpha @ H	$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i$
context	$\mathbf{c}_t$

Asosiy g'oya

Vaznlangan yig'indi NumPyda bitta vektor–matritsa ko'paytma sifatida yoziladi:

$$\alpha_t^\top \mathbf{H} = \mathbf{c}_t.$$

Vaznlar $\alpha_{t,i}$ ni qanday topish keyingi bo'limda ko'riladi.

Attention tuzog'i: vaznlar yig'indisi 1 bo'lishi shart

Normallanmagan vaznlar

Agar attention vaznlari $\alpha_{t,i}$ manfiy bo'lsa yoki yig'indisi 1 ga teng bo'lmasa, kontekst

$$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i$$

oddiy vaznlangan o'rtacha bo'lmay qoladi.

Natijada kontekstning masshtabi nazoratsiz o'zgarishi mumkin.

Yechim: softmax

Avval moslik ballari, ya'ni **energy scores** $e_{t,i}$ hisoblanadi. Keyin ular softmax orqali vaznga aylantiriladi:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}.$$

Shunda avtomatik ravishda:

$$\sum_i \alpha_{t,i} = 1, \quad \alpha_{t,i} \geq 0.$$

Softmax attention vaznlarini ehtimollik taqsimotiga aylantiradi: katta moslik balli manba holati kattaroq vazn oladi, lekin barcha vaznlar birgalikda 1 ga teng bo'lib qoladi.

Prezentatsiya rejasi

- 1 Kirish: bir tildan boshqasiga tarjima qilamiz
- 2 Enkoder-Dekoder va bottleneck
- 3 Attention: dekodeer ortga qaraydi
- 4 Attention hisoblash: query, key, value**
- 5 Tarjimani BLEU bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: so'rov kalitlarga mos kelganini izlaydi

Attention — qidiruvga o'xshaydi:

- **So'rov** (*query*) q_t — dekoderning joriy ehtiyoji: *“hozir nima kerak?”*
- **Kalitlar** (*keys*) k_i — manba tokenlarining belgisi: *“manbada nima bor?”*
- **Qiymatlar** (*values*) v_i — kontekstga qo'shiladigan axborot: *“nima qaytariladi?”*

q_t qaysi k_i bilan ko'proq mos kelsa, o'sha v_i kattaroq vazn bilan kontekstga kiradi.

Moslikdan vaznga

Avval so'rov va kalit orasidagi moslik balli hisoblanadi:

$$e_{t,i} = \text{score}(q_t, k_i).$$

Keyin softmax bu ballarni vaznlarga aylantiradi:

$$\alpha_{t,i} = \text{softmax}_i(e_{t,i}).$$

Oxirida kontekst:

$$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{v}_i.$$

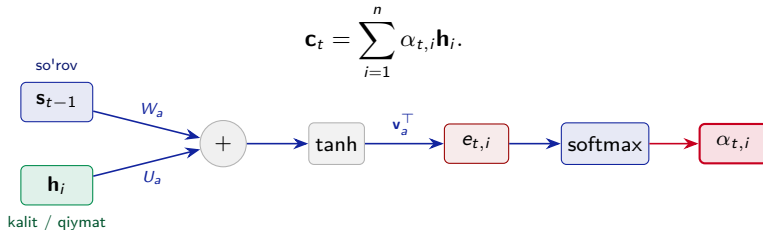
Seq2Seq attentionda k_i va v_i odatda enkoder holatlaridan olinadi; Q/K/V yozuvi esa bu jarayonni umumiy attention tili bilan ifodalaydi.

Attention (additive, Bahdanau): rasmiy ta'rif

$$e_{t,i} = \mathbf{v}_a^\top \tanh(W_a \mathbf{s}_{t-1} + U_a \mathbf{h}_i), \quad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})}.$$

bunda: $\mathbf{s}_{t-1}$ — dekoderning oldingi holati, ya'ni so'rov (*query*); $\mathbf{h}_i$ — i -enkoder holati. Bu sodda Seq2Seq attentionda $\mathbf{h}_i$ kalit (*key*) va qiymat (*value*) vazifasini bajaradi; $e_{t,i}$ — moslik balli (*energy score*); $\alpha_{t,i}$ — attention vazni; $W_a, U_a, \mathbf{v}_a$ — o'rganiladigan parametrlar.

So'ng kontekst vektori vaznlangan yig'indi orqali olinadi:



Chiziqli proyeksiyalar qo'shiladi (*additive*) → tanh → moslik balli $e_{t,i}$ → softmax → attention vazni $\alpha_{t,i}$.

Keltirib chiqarish: energiyadan vaznlarga

❶ 1-qadam. Moslik ballini hisoblaymiz.

Har bir enkoder holati uchun dekoder so'rovi bilan moslik baholanadi:

$$e_{t,i} = \text{score}(\mathbf{s}_{t-1}, \mathbf{h}_i).$$

Masalan, Bahdanau attentionda bu additive-tanh score orqali olinadi.

❷ 2-qadam. Ballarni vaznga aylantirish kerak. $e_{t,i}$ lar ixtiyoriy haqiqiy sonlar bo'lishi mumkin. Attention vaznlari esa ehtimollik taqsimoti kabi bo'lishi kerak:

$$\alpha_{t,i} \geq 0, \quad \sum_{i=1}^n \alpha_{t,i} = 1.$$

❸ 3-qadam. Softmax normalizatsiya qiladi.

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})}.$$

Xulosa

$e_{t,i}$ — “qanchalik mos?” degan moslik balli (*energy score*). Softmax uni attention vazniga, ya'ni “qanchalik e'tibor berish kerak?” degan ulushga aylantiradi. Keyingi slaydda buni qo'lda hisoblaymiz.

Qo'lda hisob: attention vaznlari $\alpha = [0.245, 0.090, 0.665]$

$$\exp(2) \approx 7.389, \quad \exp(1) \approx 2.718, \quad \exp(3) \approx 20.086.$$

Softmax hisoblaymiz:

$$Z = 7.389 + 2.718 + 20.086 = 30.193,$$

$$\alpha = \frac{[7.389, 2.718, 20.086]}{30.193},$$
$$\approx [0.245, 0.090, 0.665].$$



EkspONENTA eng katta energiyani nisbatan kuchaytiradi.

Talqin

Eng katta energiya $e_3 = 3$ bo'lgani uchun eng katta attention vazni ham uchinchi holatga tushdi: $\alpha_3 \approx 0.665$.

Demak, dekoder shu manba holatiga ko'proq e'tibor beradi.

Tekshirish

```
np.allclose(alpha,  
[0.245, 0.090, 0.665],  
atol=1e-3)
```

Sizning vazifangiz: kontekst vektorini toping

Savol

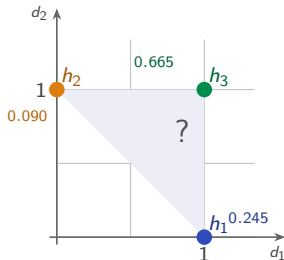
Yuqoridagi attention vaznlari va enkoder holatlari berilgan:

$$\alpha = [0.245, 0.090, 0.665], \quad h_1 = [1, 0], \quad h_2 = [0, 1], \quad h_3 = [1, 1].$$

$$\mathbf{c} = \sum_i \alpha_i h_i = ?$$

Har bir holatga mos vazn berilgan.

$\mathbf{c}$ vektorini hisoblang va grafikda **qayerga** tushishini toping. E'tibor bering: eng katta vazn h_3 da.

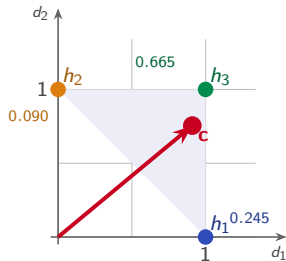


Sizning vazifangiz: kontekst vektorini toping

Javob

$$\begin{aligned} \mathbf{c} &= 0.245[1, 0] + 0.090[0, 1] + 0.665[1, 1] \\ &= [0.245 + 0.665, 0.090 + 0.665] = [\mathbf{0.910}, \mathbf{0.755}]. \end{aligned}$$

Eng katta vazn h_3 ga berilgan ($\alpha_3 = 0.665$), shu bois $\mathbf{c}$ nuqta h_3 ga yaqin joylashadi. Bu dekodeur uchun **moslashtirilgan kontekstdir**.



Vazn 0.665 h_3 da $\Rightarrow \mathbf{c}$ h_3 tomonga tortiladi.

Kod $\leftrightarrow$ formula: attention to'liq

```
import numpy as np

np.set_printoptions(precision=3, suppress=True)

def attention(energies, H):
    e = np.asarray(energies, dtype=float)

    # softmax uchun sonli barqarorlashtirish
    e = e - e.max()

    exp_e = np.exp(e)
    alpha = exp_e / exp_e.sum() # alpha = softmax(e)

    context = alpha @ H # c = sum_i alpha_i h_i
    return alpha, context

H = np.array([[1, 0], # h_1
              [0, 1], # h_2
              [1, 1]]) # h_3

alpha, c = attention([2.0, 1.0, 3.0], H)

print(alpha) # [0.245 0.09 0.665]
print(c) # [0.91 0.755]
```

Kod $\rightarrow$ formula:

Kod	Formula / ma'no
energies	$\mathbf{e}_t = [e_{t,1}, e_{t,2}, e_{t,3}]$
<code>e-e.max()</code>	softmaxni barqarorlashtirish
alpha	$\alpha_t = \text{softmax}(\mathbf{e}_t)$
<code>alpha @ H</code>	$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i$

Asosiy zanjir

Koddagi jarayon:

energiya $\rightarrow$ softmax $\rightarrow$ kontekst.

Bahdanau attentionda energiyalar maxsus additive score orqali hisoblanadi; qolgan mantiq shu.

Q/K/V: attentionning umumiy ko'rinishi

Kutubxona analogiyasi:

- **Query (so'rov)** $\mathbf{q}_t$ — *savol*: “hozir nima kerak?” dekoderning t -qadamdagi holati.
- **Key (kalit)** $\mathbf{k}_i$ — *kitob yorlig'i*: manba tokenining moslik belgisi.
- **Value (qiymat)** $\mathbf{v}_i$ — *kitob mazmuni*: kontekstga olinadigan axborot.

So'rov qaysi kalitga ko'proq mos kelsa, o'sha qiymat kontekstga kattaroq vazn bilan kiradi.

$$\text{score}_{t,i} = \mathbf{q}_t^\top \mathbf{k}_i \quad (\text{dot-product})$$

$$\alpha_{t,i} = \text{softmax}_i(\text{score}_{t,i})$$

$$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{v}_i$$

Bahdanau attention avvalgi slaydda additive score ishlatdi; dot-product esa score hisoblashning boshqa varianti. Bu darsda $\mathbf{q}_t$ — dekode holati, $\mathbf{k}_i, \mathbf{v}_i$ — enkoder holatlaridan olinadi. Transformerda esa odatda $\mathbf{q}_t^\top \mathbf{k}_i / \sqrt{d_k}$ ko'rinishi ishlatiladi.

Q/K/V tuzog'i: softmax barqarorligi

Katta energiyalar: overflow xavfi

Softmaxda $\exp(e_i)$ hisoblanadi. Agar e_i juda katta bo'lsa, sonli **overflow** yuz berishi mumkin.

Barqaror yechim:

$$\tilde{e}_i = e_i - \max_j e_j.$$

Bu natijani o'zgartirmaydi:

$$\text{softmax}(e) = \text{softmax}(e - \max(e)).$$

Nima uchun ishlaydi?

Softmax barcha energiyalarga bir xil son qo'shilsa yoki ayrilsa, nisbiy ulushlarni saqlaydi.

Shuning uchun $\max(e)$ ni ayirish faqat hisobni barqaror qiladi, attention vaznlari esa o'zgarmaydi.

Har dekoder qadamida n ta enkoder holati bilan moslik hisoblanadi: bitta qadam uchun $O(n)$, butun chiqish uzunligi m bo'lsa $O(mn)$.

Transformerda bunday moslik hisoblari matritsa amallari orqali parallel bajariladi.

Prezentatsiya rejasi

- 1 Kirish: bir tildan boshqasiga tarjima qilamiz
- 2 Enkoder-Dekoder va bottleneck
- 3 Attention: dekodeer ortga qaraydi
- 4 Attention hisoblash: query, key, value
- 5 Tarjimani BLEU bilan baholash**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: tarjima namunaviy tarjimaga qanchalik mos?

Tarjimani baholash qiyin: bitta gapning bir nechta to'g'ri tarjimasi bo'lishi mumkin.

BLEU avtomatik baholash metrikasi: mashina tarjimasidagi **n-gramlar** namunaviy tarjimada (*reference translation*) qanchalik uchrashini tekshiradi.

Yuqori BLEU $\Rightarrow$ reference bilan n-gram mosligi yuqori.

Asosiy g'oya

BLEU **aniqlikka** (*precision*) asoslanadi: nom-zod tarjimadagi (*hypothesis*) n-gramlarning nechtasi reference tarjimada bor?

Juda qisqa tarjimalarni jazolash uchun **brevity penalty** (**qisqalik jazosi**) ishlatiladi.

BLEU tarjimaning to'liq ma'nosini emas, asosan reference bilan yuzaki n-gram mosligini baholaydi. Shuning uchun u foydali, lekin mukammal semantik baho emas.

BLEU: rasmiy ta'rif

$$p_n = \frac{\text{clipped mos } n\text{-gramlar}}{\text{nomzod tarjimadagi } n\text{-gramlar}}, \quad \text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \ln p_n\right).$$

bunda: p_n — n -gram aniqligi; *hypothesis* — nomzod tarjima; *reference* — namunaviy tarjima; *clipping* — bir n -gram reference da necha marta uchrasa, ko'pi bilan shuncha hisoblash; w_n — vaznlar, odatda $w_n = 1/N$; BP — *brevity penalty* (qisqalik jazosi).

BLEU-1 faqat unigramni hisoblaydi:

$$\text{BLEU-1} = \text{BP} \cdot p_1.$$



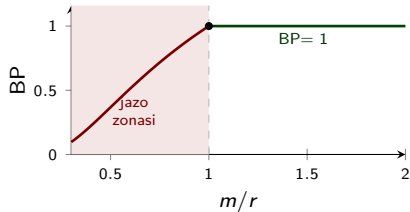
BLEU ikki narsani birlashtiradi: n -gram **aniqligi** tarjima namunaviy tarjimaga qanchalik o'xshashligini, BP esa tarjimaning haddan tashqari qisqa emasligini nazorat qiladi.

Keltirib chiqarish: nega aniqlik + brevity penalty?

- 1 **Aniqlik:** nomzod tarjimadagi (*hypothesis*) n-gramlar namunaviy tarjimada (*reference*) bormi? Bu n-gram mosligini o'lchaydi.
- 2 **Muammo:** juda qisqa tarjima ba'zan yuqori aniqlik olishi mumkin. Masalan, bitta mos so'z chiqsa, $p_1 = 1$ bo'lib qoladi, lekin tarjima to'liq emas.
- 3 **Yechim: brevity penalty (qisqalik jazosi)** juda qisqa tarjimani jazolaydi.

$$BP = \begin{cases} 1, & m \geq r, \\ e^{1-r/m}, & m < r, \end{cases}$$

bu yerda m — nomzod tarjima uzunligi,
 r — reference uzunligi.



Xulosa

BLEU n-gram **aniqligi** va **qisqalik jazosini** birlashtiradi: tarjima referencega mos bo'lishi bilan birga, haddan tashqari qisqa ham bo'lmasligi kerak.

Qo'lda misol: BLEU-1 = 0.667

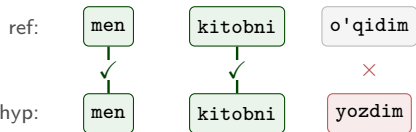
Reference (namunaviy tarjima): men kitobni o'qidim

(3 so'z)

Hypothesis (nomzod tarjima): men kitobni yozdim

(3 so'z)

Unigram aniqligi:



2 ta mos unigram ÷ 3 ta nomzod so'z:

$$p_1 = \frac{2}{3}, \quad BP = 1 \quad (3 \geq 3)$$

$$BLEU-1 = 1 \cdot \frac{2}{3} = \mathbf{0.667}$$

Talqin

Nomzod tarjimada men va kitobni so'zlari reference bilan mos tushdi.

yozdim esa o'qidim bilan mos emas.

Uzunliklar teng bo'lgani uchun qisqalik jazosi yo'q: BP = 1.

Eslatma

BLEU-1 faqat unigramlarni hisoblaydi. To'liq BLEU esa odatda bigram, trigram va 4-gram mosliklarini ham hisobga oladi.

Sizning vazifangiz: boshqa nomzod tarjima uchun BLEU-1

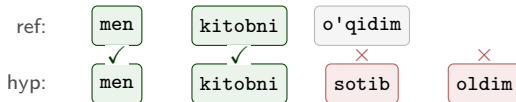
Savol

Reference (namunaviy tarjima): men kitobni o'qidim (3 so'z)

Hypothesis (nomzod tarjima): men kitobni sotib oldim (4 so'z)

Mos unigramlar: men, kitobni (2 ta)

$$p_1 = ? \quad (\text{BP} = 1, \text{ chunki } 4 \geq 3)$$



Yashil — mos so'zlar, qizil — mos emas.

Sizning vazifangiz: boshqa nomzod tarjima uchun BLEU-1

Javob

$$p_1 = \frac{2}{4} = 0.5, \quad \text{BLEU-1} = 1 \cdot 0.5 = \mathbf{0.5}.$$

Maxraj — nomzod tarjima uzunligi (4 so'z). Bu tarjima reference'dan uzunroq bo'lgani uchun BP= 1, ammo faqat 2 so'z mos kelgani uchun unigram aniqligi 0.5 ga tushadi.

	Mos so'zlar	p_1
Nomzod (4 so'z)	2 ta	$2/4 = 0.5$

Ko'proq mos so'z $\Rightarrow$ yuqoriroq p_1 ; ortiqcha noto'g'ri so'z esa aniqlikni pasaytiradi.

Kod ↔ formula: BLEU-1

```
from collections import Counter
from math import exp

def bleu1(ref, hyp):
    if len(hyp) == 0:
        return 0.0

    ref_c = Counter(ref)
    hyp_c = Counter(hyp)

    # clipped unigram matches
    matches = sum(
        min(cnt, ref_c[w])
        for w, cnt in hyp_c.items()
    )

    p1 = matches / len(hyp)

    bp = 1.0 if len(hyp) >= len(ref) else \
        exp(1 - len(ref) / len(hyp))

    return bp * p1

r = "men kitobni o'qidim".split()
h = "men kitobni yozdim".split()

print(round(bleu1(r, h), 3)) # 0.667
```

Kod → formula:

Kod	Formula / ma'no	
Counter	so'zlar	sonini
	sanash	
min(cnt, ref_c[w])	clipped match	
matches / len(hyp)	p_1	
bp * p1	BLEU-1	

Eslatma

Bu kod faqat **BLEU-1** ni hisoblaydi: unigram aniqligi p_1 va qisqalik jazosi BP. To'liq BLEU odatda 1–4-gram aniqliklarini geometrik o'rtachalaydi.

Formula

$$\text{BLEU-1} = \text{BP} \cdot p_1$$

BLEU tuzog'i: yuqori BLEU har doim yaxshi tarjima emas

BLEU cheklavlari

- **Sinonimlarni jazolashi mumkin:** n-gram aniq mos kelmasa, ma'no yaqin bo'lsa ham ball pasayadi.
- **So'z tartibiga sezgir:** qayta tartiblangan, lekin to'g'ri tarjima pastroq baholanishi mumkin.
- **Bitta reference yetarli bo'lmasligi mumkin:** bir gapning bir nechta to'g'ri tarjimasi bo'ladi.
- **Semantikani to'liq o'lchamaydi:** BLEU asosan yuzaki n-gram mosligini baholaydi.

Amaliy talqin

BLEU — tarjima tizimlarini tez va avtomatik taqqoslash uchun foydali mezon.

Lekin yakuniy xulosa chiqarishda uni **inson bahosi** yoki ma'no-asosli neyron metrikalar bilan to'ldirish kerak.

Masalan: COMET kabi metrikalar tarjimaning semantik mosligini yaxshiroq hisobga olishga harakat qiladi.

Yuqori BLEU $\Rightarrow$ reference bilan n-gram mosligi yuqori. Lekin bu har doim “eng yaxshi ma'no” degani emas.

Prezentatsiya rejasi

- 1 Kirish: bir tildan boshqasiga tarjima qilamiz
- 2 Enkoder-Dekoder va bottleneck
- 3 Attention: dekodeer ortga qaraydi
- 4 Attention hisoblash: query, key, value
- 5 Tarjimani BLEU bilan baholash
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: SOV–SVO tartibi va attention alignment

1. SOV → SVO:

O'zbekcha:

men kitobni o'qidim

ega — to'ldiruvchi — kesim (SOV)

Inglizcha:

I read the book

ega — kesim — to'ldiruvchi (SVO)

Qiyinchilik

Dekoder ingliz tilida kesimni (read) oldinroq chiqarishi kerak. Demak, tarjimada so'z tartibi qayta tashkil qilinadi.

2. Attention alignment yordam beradi:

Alignment — chiqish tokeni qaysi manba tokeniga ko'proq mos kelishini ko'rsatadigan bog'lanish.

Masalan, dekoder read so'zini yaratganda, attention o'zbekcha o'qidim tokeniga ko'proq e'tibor berishi mumkin.

Bu token manba gap oxirida turgan bo'lsa ham, inglizcha chiqishda kesim 2-o'rinda keladi.

Asosiy g'oya

Attention dekoderga chiqish tartibidan qat'li nazar, manbaning kerakli pozitsiyasiga murojaat qilishga yordam beradi. Bu o'zbek tilidagi erkinroq so'z tartibi va qo'shimchali shakllar uchun foydali.

Sintez: bottleneckdan attentionga

Jihat	Attention Seq2Seq	qo'llanmagan	Attentionli Seq2Seq
Kontekst	bitta o'zgarmas kontekst: $\mathbf{c} = \mathbf{h}_n^{enc}$		har qadam uchun yangi kontekst: $\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i$
Uzun gap	axborot torligi kuchayadi; sifat pasayishi mumkin		kerakli manba qismlariga qayta murojaat qiladi
Alignment	aniq alignment yo'lq		$\alpha_{t,i}$ orqali qaysi manba tokeni muhimligi ko'rinadi
Hisob	sodda va arzonroq		har dekoder qadamida $O(n)$ moslik hisoblanadi

Asosiy g'oya

Attention o'zgarmas kontekst bottleneckini **yumshatadi**: dekoder har qadamda (moslik balli $\rightarrow$ softmax $\rightarrow$ vaznlangan kontekst) zanjiri orqali manbaning tegishli qismiga ko'proq e'tibor beradi.

Klassik manba: Bahdanau, Cho, Bengio (2015)

Bahdanau, D., Cho, K., Bengio, Y. (2015). **Neural Machine Translation by Jointly Learning to Align and Translate.** *ICLR 2015*.

Ahamiyati:

- Neyron mashina tarjimasida **attention/alignment** g'oyasini mashhur qildi.
- O'zgarmas kontekst $\mathbf{c} = \mathbf{h}_n^{enc}$ cheklovini yumshatdi: dekoder har qadamda manbaning tegishli qismiga qaray oladi.
- Uzun gaplarda basic encoder–decoder modeliga nisbatan tarjima sifatini yaxshilashga yordam berdi.
- Keyingi Transformer arxitekturalarida attention markaziy mexanizmga aylandi.

Muhokama savoli

O'zbek tilida so'z tartibi nisbatan erkin: *men kitobni o'qidim* va *kitobni men o'qidim* ikkalasi ham tushunarli. Attention alignment bunday tartib almashishida qanday yordam beradi? O'zgarmas kontekst nima uchun qiynaladi?

Bugungi maqsadlar bajarildi

- ① ✓ **Tushuntirdik:** enkoder–dekoder arxitekturası va **axborot torligi** (*information bottleneck*) muammosi:

$$\mathbf{c} = \mathbf{h}_n^{enc}.$$

- ② ✓ **Tushuntirdik:** attention (**e'tibor mexanizmi**) har dekoder qadamida yangi kontekst hosil qiladi:

$$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i.$$

- ③ ✓ **Hisobladik:**

$$\alpha = \text{softmax}([2, 1, 3]) = [0.245, 0.090, 0.665],$$

va kontekst:

$$\mathbf{c} = [0.910, 0.755].$$

- ④ ✓ **Hisobladik:** BLEU-1 = 0.667; bunda unigram aniqligi va **qisqalik jazosi** (*brevity penalty*) hisobga olindi.

Keyingi ma'ruza (L12): cross-attentiondan self-attentionga

Bugun: cross-attention Dekoder **enkoder** holatlariga qaraydi:

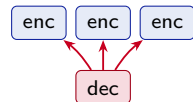
$$q_t \leftarrow \text{dekoder}, \quad k_i, v_i \leftarrow \text{enkoder}.$$

- **L12: self-attention.** Har token uchun q, k, v bir xil ketma-ketlikdan olinadi.
- Encoder self-attentionda tokenlar bir-biriga qaraydi; decoder self-attentionda esa kelajak tokenlar odatda maskalanadi.
- Rekurrent o'qish o'rniga matritsa amallari ishlatiladi $\Rightarrow$ **Transformer**.

Asosiy ko'prik

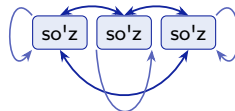
Q/K/V mexanizmi o'sha: farq shundaki, q, k, v **qaysi ketma-ketlikdan** olinishi o'zgaradi.

cross-attention (L11)



q : dekoder $\rightarrow k, v$: enkoder

self-attention (L12)



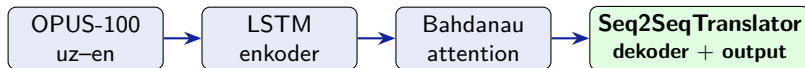
q, k, v : bitta ketma-ketlikdan (o'ziga ham)

Qisqacha

L11: dekoder tokeni butun manbaga qaraydi (ikki ketma-ketlik).

L12: token o'sha ketma-ketlikdagi barcha tokenlarga qaraydi.

11-Amaliyot: attentionli neyro-tarjimon quriladi



11-amaliyotda qurasiz:

- Seq2SeqTranslator: enkoder-dekoder + attention.
- OPUS-100 uz-en juftligida o'qitish.
- Tarjimani BLEU bilan baholash.
- Softmax va attention vaznlari α ni tekshirish.

Birinchi assert: softmax testi

```
alpha = softmax([2.0, 1.0, 3.0])
assert np.allclose(
    alpha, [0.245, 0.090, 0.665],
    atol=1e-3)
```

Bu assert **o'qitilgan model vazni** uchun emas; ma'ruzadagi softmax hisobini kodda tekshirish uchun.

- ❶ Bahdanau, D., Cho, K., Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015.
- ❷ Sutskever, I., Vinyals, O., Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*. NIPS/NeurIPS 2014.
- ❸ Luong, M.-T., Pham, H., Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. EMNLP 2015.
- ❹ Papineni, K., Roukos, S., Ward, T., Zhu, W.-J. (2002). *BLEU: a Method for Automatic Evaluation of Machine Translation*. ACL 2002.
- ❺ Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS 2017.
- ❻ NLTK hujjatlari: `nltk.translate.bleu_score` moduli (*sentence_bleu*, *corpus_bleu*, *smoothing*).

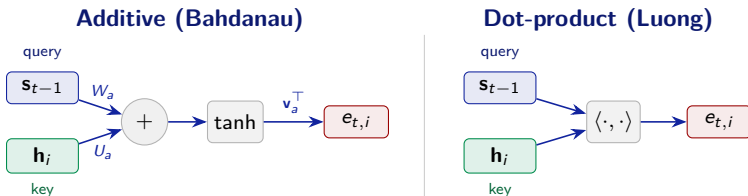
Ilova S1: additive vs dot-product attention

Moslik energiyasi $\mathbf{e}_{t,i}$ ni hisoblashning ikki keng tarqalgan usuli:

- **Additive** (Bahdanau): $e_{t,i} = \mathbf{v}_a^\top \tanh(W_a \mathbf{s}_{t-1} + U_a \mathbf{h}_i)$ — kichik o'rganiladigan MLP.
- **Dot-product** (Luongning *dot* varianti): $e_{t,i} = \mathbf{s}_{t-1}^\top \mathbf{h}_i$ — to'g'ridan-to'g'ri skalyar ko'paytma.

Transformer (L12) shu g'oyani **masshtablangan dot-product attention** bilan davom ettiradi:

$$\text{score}(q, k) = \frac{q^\top k}{\sqrt{d_k}}.$$



Additive — ko'proq parametr, moslashuvchan; dot-product — sodda va tez.

Ilova S2: to'liq BLEU (n -gram va clipping)

To'liq BLEU bir nechta n -gram aniqliklarini geometrik o'rtachalaydi:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \ln p_n\right), \quad \text{odatda } N = 4, w_n = \frac{1}{N}.$$

Bir n -gram **reference** da necha marta uchrasa, hypothesisdagi mosliklar **ko'pi bilan shuncha** hisoblanadi. Bu takroriy so'z bilan sun'iy ravishda yuqori aniqlik olishni cheklaydi.

Misol:

- Hypothesis: the the the
- Reference: the

$$\text{xom moslik} = 3 \Rightarrow p_1^{\text{oddiy}} = \frac{3}{3} = 1$$

hyp: the the the xom moslik = 3

ref: the ruxsat etilgan moslik: 1

$$\min(3, 1) = 1 \Rightarrow p_1 = \frac{1}{3}$$